Universidad Técnica Estatal de Quevedo

Facultad de Ciencias de la Computación
Carrera de Software

Aplicaciones Distribuidas (ISR-701)
FORM-FB-01: Fichas bibliográficas
Fundamentos de sistemas distribuidos y arquitecturas distribuidas

Estudiante: Reinoso Vélez Eduardo David
Nivel y paralelo: Séptimo nivel, paralelo A
Fecha: 6 de noviembre de 2026
Repositorio: `https://github.com/ereinosov/app-dist-form-fb-01`

Sistema elegido para TA-IND-01: Apache Cassandra, comparado con Amazon DynamoDB.

Ficha 1. Ferreira et al. (2024): niveles de consistencia en NoSQL

Referencia (IEEE)

S. Ferreira et al., «Impacts of data consistency levels in cloud-based NoSQL for data-intensive applications,» *Journal of Cloud Computing*, vol. 13, 2024, Art. 158. DOI: `10.1186/s13677-024-00716-7`

DOI verificado

10.1186/s13677-024-00716-7

Resumen (100 palabras)

Ferreira y su equipo quieren saber cuánto rendimiento se pierde al exigir más consistencia a una base NoSQL (Not only SQL, no relacional). Prueban Cassandra, MongoDB y Redis en Google Cloud, cada una por separado, con la herramienta de carga Locust y entre 1 y 10 usuarios simultáneos. Cassandra corre con tres nodos y factor de replicación 3, y comparan el nivel ONE (basta la respuesta de una réplica) con ALL (responden todas). Lo repiten con los nodos en una región y en varias. El resultado es claro: la consistencia fuerte baja el rendimiento y alarga las respuestas, y la distancia entre regiones agrava el efecto.

Dato concreto (con página o sección)

En la Tabla 8 (p. 14), pasar de consistencia débil a fuerte en Cassandra con réplicas en varias regiones reduce el rendimiento de escritura un 97,54 % y sube el tiempo de respuesta un 6.028,34 %. En una sola región, las lecturas pierden un 62,30 % de rendimiento y tardan un 175,15 % más.

Nota personal

Esta fuente me da números concretos para la parte de CAP (consistencia, disponibilidad y tolerancia a particiones) y PACELC (si hay partición, disponibilidad o consistencia; si no, latencia o consistencia) de TA-IND-01. Tengo presente que su nivel fuerte es ALL y que solo un nodo recibió las peticiones, así que no puedo usar sus cifras como si fueran de QUORUM (mayoría de réplicas).

Ficha 2. Rehman, Aguiar y Barraca (2022): tolerancia a fallos en la nube

Referencia (IEEE)

A. U. Rehman et al., «Fault-Tolerance in the Scope of Cloud Computing,» *IEEE Access*, vol. 10, págs. 63 422-63 441, 2022. DOI: 10.1109/ACCESS.2022.3182211

DOI verificado

10.1109/ACCESS.2022.3182211

Resumen (100 palabras)

Rehman, Aguiar y Barraca revisan más de cien trabajos sobre tolerancia a fallos en la nube. Empiezan fijando tres términos: falta (la causa), error (el estado que se desvía de lo esperado) y fallo (cuando el sistema deja de cumplir su función). Después reparten las técnicas en reactivas, que actúan cuando el problema ya ocurrió, y proactivas, que intentan adelantarse. Relacionan métricas de componente y de sistema, comparan arquitecturas existentes y cierran con temas abiertos: nube, niebla y borde, aprendizaje automático y consumo de energía. Su conclusión es que combinar ambos enfoques probablemente rinda más que elegir uno solo.

Dato concreto (con página o sección)

Secciones IV.F y IV.G. Entre las técnicas reactivas citan checkpoint/restart (guardar el estado y reiniciar desde ahí), replicación, migración de tareas y reintento. Entre las proactivas, autorreparación, migración preventiva, rejuvenecimiento de software y balanceo de carga. La sección VII advierte que las proactivas pueden costar más por el monitoreo continuo.

Nota personal

Me sirve de marco para ordenar lo que hacen Cassandra y DynamoDB ante un fallo: la replicación y los reintentos quedan del lado reactivo, y con esta clasificación puedo decirlo con las mismas palabras en la sección de tolerancia a fallos de TA-IND-01.

Ficha 3. Kumari y Kaur (2021): revisión de tolerancia a fallos

Referencia (IEEE)

P. Kumari y P. Kaur, «A survey of fault tolerance in cloud computing,» *Journal of King Saud University - Computer and Information Sciences*, vol. 33, n.º 10, págs. 1159-1176, 2021, ISSN: 1319-1578. DOI: 10.1016/j.jksuci.2018.09.021

DOI verificado

10.1016/j.jksuci.2018.09.021

Resumen (100 palabras)

Kumari y Kaur escriben una revisión sobre cómo mantener un servicio en la nube cuando algo falla. Primero describen la infraestructura y los modelos de despliegue y de servicio, y luego repasan los enfoques que ya existían en sistemas distribuidos. El núcleo es la clasificación en técnicas proactivas y reactivas, con ejemplos como la replicación, el reintento y la migración, y un apartado sobre su uso en aplicaciones basadas en la nube. Proponen una tabla de métricas para evaluarlas, con fiabilidad, disponibilidad, tiempo de respuesta, rendimiento y costo. Cierran con líneas de investigación futura, entre ellas el aprendizaje profundo.

Dato concreto (con página o sección)

Secciones 4.2 y 4.3 y Tabla 3. Proactivas: autorreparación, rejuvenecimiento de software, migración preventiva y balanceo de carga. Reactivas: checkpointing, migración de tareas, replicación, reintento y reenvío de tareas. La Tabla 3 reúne las métricas para evaluar la tolerancia a fallos: rendimiento, tiempo de respuesta, fiabilidad, disponibilidad y costo, entre otras.

Nota personal

La clasificación coincide en gran parte con la de la ficha 2, pero su lista de métricas (Tabla 3) es lo más aprovechable: la usaré para decidir con qué criterios comparar Cassandra y DynamoDB en TA-IND-01.

Ficha 4. Huang et al. (2022): fallos metaestables

Referencia (IEEE)

L. Huang et al., «Metastable Failures in the Wild,» en *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, Carlsbad, CA, USA: USENIX Association, jul. de 2022, págs. 73-90, ISBN: 978-1-939133-28-1. dirección: `https://www.usenix.org/conference/osdi22/presentation/huang-lexiang`

ISBN verificado

978-1-939133-28-1 (actas de OSDI 22; el artículo no tiene DOI)

Resumen (100 palabras)

Huang y sus coautores estudian los fallos metaestables: una sobrecarga que sigue aunque ya haya desaparecido lo que la provocó. Para ver cómo ocurren en la práctica, revisan informes públicos de incidentes de empresas grandes y pequeñas. Separan el disparador (un pico de carga o una pérdida de capacidad) de lo que mantiene el problema, como los reintentos o la recolección de basura (liberación automática de memoria). Reproducen tres casos en aplicaciones de ejemplo de código abierto. Concluyen que estos fallos aparecen en las caídas más graves y que los reintentos son el factor de mantenimiento más frecuente.

Dato concreto (con página o sección)

Sección 2.2 (p. 75): los reintentos son el efecto que más veces mantiene el fallo, presente en más de la mitad de los incidentes. Resumen (p. 73): al menos 4 de las 15 caídas mayores de AWS (Amazon Web Services) en la última década fueron metaestables.

Nota personal

Me da un caso real de fallo para la sección de tolerancia a fallos de TA-IND-01 y una advertencia útil: los reintentos de los clientes pueden empeorar la sobrecarga de un clúster, algo que tendré en cuenta al hablar de Cassandra bajo carga.

Ficha 5. Elhemali et al. (2022): Amazon DynamoDB

Referencia (IEEE)

M. Elhemali et al., «Amazon DynamoDB: A Scalable, Predictably Performant, and Fully Managed NoSQL Database Service,» en *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, Carlsbad, CA, USA: USENIX Association, jul. de 2022, págs. 1037-1048, ISBN: 978-1-939133-29-8. dirección: `https://www.usenix.org/conference/atc22/presentation/elhemali`

ISBN verificado

978-1-939133-29-8 (actas de USENIX ATC 22; el artículo no tiene DOI)

Resumen (100 palabras)

Elhemali y su equipo de AWS cuentan cómo operan DynamoDB a gran escala y cómo cambió su arquitectura desde 2012. Presentan seis propiedades del servicio: gestión completa, multiinquilino, escala sin límite, rendimiento predecible, alta disponibilidad y uso flexible. Cada partición se copia en varias zonas de disponibilidad y elige líder con Multi-Paxos (protocolo de consenso). Solo el líder atiende escrituras y lecturas fuertes. El texto explica también cómo manejan fallos silenciosos, particiones calientes y despliegues, y da cifras reales, como un pico de 89,2 millones de peticiones por segundo en el Prime Day de 2021.

Dato concreto (con página o sección)

p. 1040: una escritura se confirma cuando un quórum (mayoría) de réplicas guarda el registro; solo el líder atiende escrituras y lecturas fuertemente consistentes, mientras que cualquier réplica puede atender lecturas eventualmente consistentes. p. 1045: un quórum de escritura sano son 2 de 3 réplicas en zonas de disponibilidad distintas.

Nota personal

Es la base de lo que voy a contar de DynamoDB en la comparación de TA-IND-01. Su esquema de un líder por partición contrasta con Cassandra, que no tiene nodo líder, y ahí se explica buena parte de la diferencia en consistencia.

Referencias

[1] S. Ferreira, J. Mendonça, B. Nogueira, W. Tiengo y E. Andrade, «Impacts of data consistency levels in cloud-based NoSQL for data-intensive applications,» *Journal of Cloud Computing*, vol. 13, 2024, Art. 158. DOI: 10.1186/s13677-024-00716-7

[2] A. U. Rehman, R. L. Aguiar y J. P. Barraca, «Fault-Tolerance in the Scope of Cloud Computing,» *IEEE Access*, vol. 10, págs. 63 422-63 441, 2022. DOI: 10.1109/ACCESS.2022.3182211

[3] P. Kumari y P. Kaur, «A survey of fault tolerance in cloud computing,» *Journal of King Saud University - Computer and Information Sciences*, vol. 33, n.º 10, págs. 1159-1176, 2021, ISSN: 1319-1578. DOI: 10.1016/j.jksuci.2018.09.021

[4] L. Huang et al., «Metastable Failures in the Wild,» en *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, Carlsbad, CA, USA: USENIX Association, jul. de 2022, págs. 73-90, ISBN: 978-1-939133-28-1. dirección: https://www.usenix.org/conference/osdi22/presentation/huang-lexiang

[5] M. Elhemali et al., «Amazon DynamoDB: A Scalable, Predictably Performant, and Fully Managed NoSQL Database Service,» en *2022 USENIX Annual Technical Conference (USENIX ATC 22)*, Carlsbad, CA, USA: USENIX Association, jul. de 2022, págs. 1037-1048, ISBN: 978-1-939133-29-8. dirección: https://www.usenix.org/conference/atc22/presentation/elhemali

Declaración de uso de inteligencia artificial

Herramienta: Claude (Anthropic), modelo Claude Sonnet 5.5.

Para qué la usé: (1) buscar en la web fuentes candidatas sobre consistencia, tolerancia a fallos y comunicación en sistemas distribuidos, con Apache Cassandra como sistema elegido para TA-IND-01; (2) generar la plantilla LaTeX de este documento y el esqueleto del archivo `refs.bib`, y (3) cotejar los datos bibliográficos de las candidatas con páginas de editoriales y repositorios.

Cómo verifiqué lo obtenido: abrí personalmente el DOI o el enlace de cada una de las cinco fuentes y comparé autores, año, título, revista o libro, volumen, número y páginas con `refs.bib`; corregí las diferencias; consulté en Scopus/SJR la indexación y el cuartil de cada revista; y leí cada fuente para localizar el dato concreto y su página.

Redacción de las fichas: la herramienta me ayudó a redactar los borradores iniciales de los resúmenes y las notas, a partir de los resúmenes de las fuentes. Yo verifiqué cada afirmación, ubiqué los datos concretos con su página y reescribí los textos con mis propias palabras. Los datos y sus páginas son los que constaté en las fuentes.